

NetQMPI: An MPI-Inspired software for programming Distributed Quantum Applications over Quantum Networks using NetQASM SDK

F. JAVIER CARDAMA¹, and TOMÁS F. PENA^{1,2}

¹Centro Singular de Investigación en Tecnoloxías Intelixentes (CITIUS), Universidade de Santiago de Compostela, Santiago de Compostela, 15782, Spain

²Departamento de Electrónica e Computación, Universidade de Santiago de Compostela, Santiago de Compostela, 15782, Spain

Corresponding author: F. Javier Cardama (email: javier.cardama@usc.es).

This work was funded by European Union EuroHPC program with grant agreement 101194491 and by MICIU/AEI/10.13039/501100011033 with project number PCI2025-163229, the Agencia Estatal de Investigación (Spain) (PID2022-141623NB-I00), the Consellería de Cultura, Educación e Ordenación Universitaria Galician Research Center accreditation 2024–2027 ED431G-2023/04, and the European Regional Development Fund (ERDF) .

ABSTRACT Distributed Quantum Computing (DQC) is essential for scaling quantum algorithms beyond the limitations of monolithic NISQ devices. However, the current software ecosystem forces developers to manually orchestrate low-level network resources, such as entanglement generation (EPR pairs) and classical synchronization, leading to verbose, error-prone, and non-scalable code. This paper introduces **NetQMPI**, a high-level Python framework that adapts the Message Passing Interface (MPI) standard to the quantum domain using the Single Program Multiple Data (SPMD) paradigm. Built as a middleware over the NetQASM SDK, NetQMPI abstracts the underlying physical topology, automating network initialization and resource management through a unified `Communicator` interface. We propose semantic point-to-point primitives and novel collective operations—such as `expose` and `unexpose`—that address the constraints of the No-Cloning Theorem by leveraging multipartite entanglement for data distribution. Our comparative analysis demonstrates that NetQMPI decouples algorithmic logic from network size, reducing the code complexity for generating an N -node GHZ state from $\mathcal{O}(N^2)$ to constant complexity $\mathcal{O}(1)$. Furthermore, the framework ensures backend agnosticism, enabling the seamless execution of high-level applications on rigorous physical simulators like NetSquid (via SquidASM) and future quantum hardware adhering to the NetQASM standard.

INDEX TERMS distributed quantum computing, high-performance computing, quantum internet, mpi, quantum networks

I. INTRODUCTION

In recent years, quantum computing has emerged as a computational paradigm capable of offering exponential speedups for problems intractable on classical machines [1], [2], such as integer factorization [3] and unstructured search [4]. However, realizing these algorithms at scale requires a substantial number of qubits and robust error correction, capabilities that remain challenging for current hardware in the noisy intermediate-scale quantum (NISQ) era [5].

To overcome these scaling limitations, distributed quantum computing (DQC) has emerged as a promising path. By interconnecting multiple quantum processing units (QPUs) via a quantum network, it is possible to create a virtual cluster capable of executing large-scale algorithms [6]–[8]. However, orchestrating computations across such a distributed infras-

tructure introduces engineering complexities [9]. Unlike classical networks, quantum networks require the management of fragile resources such as entanglement, Einstein-Podolsky-Rosen (EPR) pairs, and the tight synchronization of classical and quantum control planes [10].

Nevertheless, the transition from monolithic to distributed quantum algorithms introduces significant software engineering challenges [11]. Most existing software tools allow to researchers to program at the physical or link layer, managing elementary operations explicitly [12], [13]. This low-level approach not only increases development complexity but also couples the algorithmic logic to specific hardware topologies or simulation backends, severely limiting code portability and maintainability. In the realm of classical high-performance computing (HPC), this challenge was successfully addressed

by the message passing interface (MPI) standard, which unified distributed programming under the single program multiple data (SPMD) paradigm. While theoretical proposals for a quantum message passing interface (QMPI) exist [14], the ecosystem still lacks a functional, standardized implementation that bridges the gap between high-level algorithmic design and the execution on rigorous quantum network stacks.

A. CONTRIBUTIONS

In this work, we present **NetQMPI**, a high-level MPI-inspired Python library for programming distributed quantum applications over quantum networks. NetQMPI acts as a middleware layer over the standard NetQASM software development kit (SDK), allowing developers to write distributed quantum algorithms using high-level semantics while ensuring execution on rigorous network simulators.

The main objectives and novel contributions of this work are:

- **Adoption of the SPMD Paradigm:** We propose a shift from role-based scripting to a unified Single Program Multiple Data model. By injecting logical `rank` identifiers, NetQMPI allows a single source file to orchestrate complex protocols across N nodes, decoupling the code complexity from the network size.
- **Abstraction of Physical Resources:** We introduce the `QMPICommunicator` and semantic primitives (e.g., `qsend`, `qrecv`) that abstract the management of EPR sockets and classical control planes. This allows researchers to focus on algorithmic logic rather than the intricate details of entanglement generation and fidelity management.
- **Novel Collective Operations:** We define and implement quantum-specific collective operations, such as `expose` and `unexpose`. These primitives address the constraints of the No-Cloning Theorem by leveraging multipartite entanglement to share quantum information across the communicator without physical copying.
- **Backend-Agnostic Execution:** By targeting the standardized NetQASM instruction set architecture (ISA), NetQMPI serves as a unified interface that can execute applications on high-fidelity simulators (such as NetSquid via SquidASM) and is forward-compatible with future quantum hardware, bridging the gap between abstract algorithm design and realistic physical simulation.

This paper is organized as follows, Section II provides the necessary background on quantum networking tools through the different layers of abstraction, also explaining NetQASM SDK. Section III details the architecture of NetQMPI as well as the creation process on NetQASM SDK and the high-level semantic functions provided.. Section IV a comparative analysis of the various tools in the literature cited in the related work, and Section V concludes the paper.

II. RELATED WORK

Recent advances in quantum algorithms have driven considerable efforts in developing software tools for programming

and simulating distributed quantum systems. This section presents the most relevant tools in this domain, distinguishing between high-level abstractions and low-level physical simulators.

A. SOFTWARE TOOLS FOR PROGRAMMING DISTRIBUTED QUANTUM SYSTEMS

To manage the complexity of distributed quantum states, several frameworks have been proposed to abstract the underlying physical layer.

Interling [15], offers a high-level abstraction layer built upon the QuNetSim communication framework, designed to automate circuit partitioning and message scheduling across distributed nodes.

- *Strengths:* It simplifies distributed execution by automating circuit partitioning and teleportation scheduling, allowing monolithic circuits to be run across multiple nodes (“circuit knitting”) without manual intervention for small-scale systems.
- *Weaknesses:* It suffers from severe scalability issues, where execution time and memory usage grow exponentially with the number of qubits and nodes. Additionally, its automatic partitioner lacks advanced optimization techniques, leading to redundant teleportations and communication overhead, while imposing rigid constraints on node configurations.

QuNetSim [16], created by the Technical University of Munich, provides a high-level Python API for simulating network protocols (e.g., quantum key distribution (QKD), teleportation) without dealing with hardware physics.

- *Strengths:* It offers an extremely gentle learning curve and built-in implementations of standard protocols.
- *Weaknesses:* It is purely a simulator with no path to physical hardware execution; its abstraction level hides the resource management details (like entanglement fidelity) required for realistic system programming.

QMPI [14], proposed by Häner et al. (ETH Zurich), introduces the theoretical foundation for applying the Message Passing Interface (MPI) standard to quantum computing.

- *Strengths:* It leverages the mature MPI standard (commands like `Send`, `Recv`, `Bcast`) familiar to the HPC community.
- *Weaknesses:* There is no known implementation or functional application of this proposal, as the work is limited to defining the semantics. Furthermore, it does not address how to execute collective operations such as `Bcast` or `Allgather`; these primitives inherently imply data copying in classical MPI, which conflicts with the No-Cloning Theorem, and the proposal offers no mechanism to resolve this.

B. QUANTUM NETWORK SIMULATORS

This category encompasses tools designed to simulate any quantum network. These tools allow the execution of

platform-independent code, often relying on lower-level engines for physical accuracy.

SimulaQron [17] focuses on the distributed nature of the network. It runs a simulation where each network node is a separate process on the host machine, communicating via classical sockets to imitate a real distributed architecture.

- *Strengths*: It enforces a strict separation of memory between nodes, making it ideal for verifying client-server logic and distributed application flows.
- *Integration*: While it acts as a standalone distributed backend, it is part of the QuTech ecosystem and was a precursor to modern execution environments.

SquidASM [18] is the specialized execution tool for NetQASM applications. It acts as a bridge between the application layer and the physical layer.

- *Strengths*: It allows developers to run NetQASM routines on a simulated network with high fidelity.
- *Integration*: SquidASM is explicitly built on top of **NetSquid**. It translates the high-level instructions from the SDK into discrete events that NetSquid can process, providing a realistic noise model while maintaining a programmable interface.

C. DISCRETE-EVENT SIMULATORS

At the lowest level of abstraction, these tools model the physical transmission of qubits, time-dependent noise, and hardware components (memories, repeaters, fibers).

NetSquid [19] is the state-of-the-art platform for modeling quantum networks at the physical layer.

- *Strengths*: It handles precise timing of photon emission, fiber loss, and quantum memory decoherence. It serves as the physics engine powering higher-level tools like SquidASM.
- *Weaknesses*: The complexity of defining hardware components makes it cumbersome for general application programming; it is designed for network architects rather than algorithm developers.

SeQuENCe [20] is a customizable discrete-event simulator with a strong focus on the network layer (routing, resource management).

- *Strengths*: Excellent for studying network topology and protocol stacks and available as open-source.
- *Weaknesses*: Like NetSquid, it presents a high barrier to entry for software developers simply wanting to write quantum applications using standard programming paradigms.

D. NETQASM SDK OVERVIEW

The foundation of the work presented in this paper is the NetQASM framework [18], which can be distinguished into two main components: the Instruction Set Architecture (ISA) and the Software Development Kit (SDK).

1) NetQASM ISA

NetQASM defines a low-level, platform-independent Instruction Set Architecture designed specifically for quantum networks. It serves as a common language that abstracts the heterogeneity of underlying hardware. Programs compiled into NetQASM assembly can be executed on various backends, ranging from high-fidelity physical simulators like SquidASM (based on NetSquid) or SimulaQron. This architecture separates the quantum logic from the control hardware, allowing for hybrid quantum-classical execution.

```

1 # NetQASM Assembly (Alice Side)
2 set R0 0 # Register for Qubit ID
3 qalloc R0 # Allocate Control Qubit
4 init R0 # Initialize to |0>
5
6 # -- Entanglement Generation --
7 store 1 @0 # Store arg: number of pairs
8 store 0 @1 # Store arg: type
9 create_epr 0 1 0 @0
10 wait_all @0 # Explicit wait for hardware
11
12 set R1 1 # Manually assign ID to EPR qubit
13 cnot R0 R1 # Local CNOT
14 meas R1 R2 # Measure EPR into Register R2
15 qfree R1 # Free EPR qubit resource

```

FIGURE 1. Snippet of NetQASM ISA Assembly corresponding to Alice's operations. It highlights the low-level management of registers (e.g., R0, R1) and memory addresses (e.g., @0) required by the architecture.

2) The SDK and EPR Sockets

To facilitate programming without writing raw assembly, the NetQASM SDK provides a Python interface to write applications for specific network roles (e.g., "Alice" and "Bob"). A central abstraction in the SDK is the *EPR Socket*. By opening an EPR socket between two nodes, the programmer can request the generation of entanglement (e.g., creating Bell pairs) utilizing the underlying network link layer.

Figure 2 demonstrates the implementation of a teleport protocol using the NetQASM SDK, highlighting the steps required to transfer a qubit state from Alice to Bob.

3) Programming Complexity

While the SDK abstracts the physical generation of entanglement, it does not provide high-level primitives for qubit transmission. Crucially, to transfer a quantum state from one node to another, the generation of an EPR pair is merely the first step. The programmer must manually implement the full *Quantum Teleportation* or *Remote State Preparation* (Teledata) protocol *ad hoc*.

For example, to send a single qubit via teleportation using the SDK, the developer must explicitly:

- 1) Request entanglement via the EPR Socket and wait for completion.
- 2) Perform local gates (CNOT, Hadamard) between the payload qubit and the entangled qubit.
- 3) Measure the qubits and obtain the classical outcomes.
- 4) Send the classical bits to the receiver using a separate classical channel.

- 5) Receiver side: read these bits and apply the corresponding Pauli corrections (X or Z) to recover the state.

Alice (Control Node)

```

1 # Create a socket to send classical information
2 socket = Socket("Bob", "Alice", log_config=log_config)
3
4 # Create a EPR socket for entanglement generation
5 epr_socket = EPRSocket("Bob")
6
7 with NetQASMConnection(epr_sockets=[epr_socket]) as conn:
8     # Create a qubit to teleport
9     q = Qubit(conn)
10
11     # Create EPR pairs
12     epr = epr_socket.create_keep()[0]
13
14     # Teleport
15     q.cnot(epr)
16     q.H()
17     m1 = q.measure()
18     m2 = epr.measure()
19
20 # Send the correction information
21 m1, m2 = int(m1), int(m2)
22
23 socket.send_structured(StructuredMessage("Corrections", (
24     m1, m2)))

```

Bob (Target Node)

```

# Create a classical socket
socket = Socket("Alice", "Bob")

# Create a EPR socket
epr_socket = EPRSocket("Alice")

with NetQASMConnection(epr_sockets=[epr_socket]) as conn:
    # 1. Receive EPR qubit
    epr = epr_socket.recv_keep()[0]

    # 2. Flush to sync
    conn.flush()

    # 3. Get and apply the corrections
    m1, m2 = socket.recv_structured().payload

    if m2 == 1:
        epr.X()

    if m1 == 1:
        epr.Z()

    # 4. Final flush to sync
    conn.flush()

```

FIGURE 2. Implementation of a teleport protocol using NetQASM SDK. The developer is responsible for manually orchestrating the EPR socket creation, classical message passing, and synchronization (flush).

This requirement to manually orchestrate classical and quantum synchronization for every transfer leads to verbose and error-prone code. This limitation is the primary motivation for NetQMPI, which encapsulates these complex workflows into simple, atomic communication primitives.

III. NETQMPI PROPOSAL

This section introduces NetQMPI, a high-level Python library designed to simplify the development of distributed quantum applications. Inspired by the standardized Message Passing Interface (MPI) used in classical High-Performance Computing (HPC), NetQMPI adopts the Single Program Multiple Data (SPMD) paradigm. This allows developers to write a single code, platform-independent source code that orchestrates quantum operations across multiple network nodes without managing the underlying physical entanglement resources manually.

A. SOFTWARE STACK

The architectural position of NetQMPI within the quantum network software ecosystem is illustrated in Figure 3. NetQMPI acts as a middleware layer that sits directly on top of the NetQASM SDK.

A primary objective of this architecture is to bridge the gap between high-level algorithmic design and platform-specific execution. By abstracting the NetQASM SDK, NetQMPI hides the complexity of connection management and explicit instruction compilation. The user application interacts solely with NetQMPI primitives (e.g., `qsend`, `qrecv`), which are then translated at runtime into the corresponding NetQASM subroutines—handling `EPRSocket` lifecycle,

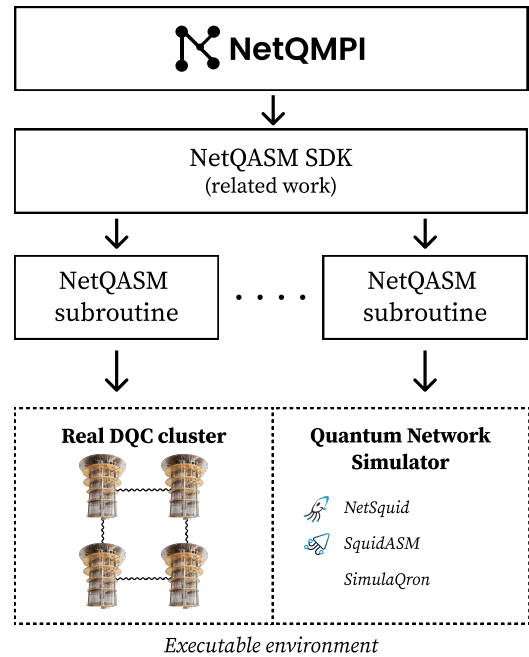


FIGURE 3. The NetQMPI software stack. The library wraps the verbose NetQASM SDK primitives, providing a unified API for the application layer while relying on the standard NetQASM ISA to interface with backend simulators like SquidASM or SimulaQron.

NetQASMConnection management, and classical control flow transparently.

Because these primitives compile down to the standardized NetQASM Instruction Set Architecture (ISA), NetQMPI is inherently backend-agnostic. This abstraction allows devel-

opers to write code that is completely decoupled from the underlying infrastructure, enabling the execution of the same source code on both *simulated networks* and *physical quantum networks* without modification.

Furthermore, this design leverages the robust capabilities of the ecosystem. Specifically, it allows users to utilize high-fidelity simulators like NetSquid (via the SquidASM interface) without facing its steep learning curve. While NetSquid is the state-of-the-art for modeling physical noise and network delays, programming it directly for application development is notoriously complex. NetQMPI enables researchers to validate algorithms using NetSquid's realistic physics engine while maintaining a high-level, abstract programming model.

B. CONSTRUCTION OF NETQMPI FROM NETQASM SDK

NetQMPI¹ is constructed as a wrapper that fundamentally alters the workflow of the NetQASM SDK. To achieve the transition from the role-based scripts required by the SDK to the Single Program Multiple Data (SPMD) paradigm, NetQMPI relies on a specific internal architecture. Figure 4 illustrates the organization of the source code.

The core orchestration is handled by the `external.py` module, which manages the multiprocess environment mapping. The network state and automated entanglement generation are encapsulated within the `communicator` package. Finally, the specific communication logic is organized under `primitives`, divided into `p2p` and `collective` modules, whose specific implementations will be detailed in Subsections III-C and III-D, respectively.

```
1 netqmpi/
2 |-- sdk/
3 |   |-- external.py
4 |   |-- communicator/
5 |   |   |-- communicator.py
6 |   |   |-- primitives/
7 |   |   |-- collective/
8 |   |-- p2p/
```

FIGURE 4. Directory structure of the NetQMPI library. The `external.py` module orchestrates the multiprocess execution, while `communicator.py` manages the automated entanglement generation.

1) Single Program Architecture

In the standard NetQASM SDK, the developer is required to write N separate source files (e.g., `app_alice.py`, `app_bob.py`), one for each node in the network. NetQMPI unifies this into a single file following the SPMD paradigm.

To enable this, NetQMPI utilizes the abstraction of the *rank*. For readers unfamiliar with HPC terminologies, the rank is simply a unique integer identifier assigned to each process in the distributed network, ranging from 0 to $size - 1$. This value is injected automatically by the library into the user's code at runtime, allowing the developer to define the logic for all nodes in a single function signature.

¹The complete source code is available at: <https://github.com/netqir/netqmpi>

The user script must accept *rank* and *size* as arguments, which are then used to initialize the `QMPICommunicator`, as shown in Figure 5.

```
1 from netqmpi.sdk.communicator import QMPICommunicator
2
3 def main(app_config=None, rank=0, size=1):
4     # The rank is injected by NetQMPI
5     # and used to initialize the local communicator
6     COMM_WORLD = QMPICommunicator(rank, size, app_config)
7     print(f"Hello, rank={rank} of {size} processes")
```

FIGURE 5. Example of the rank and size variable injection with the global communicator creation.

When the user executes this script using the Command Line Interface (CLI), the `cli.py` module parses the `-n` argument (number of processes) and triggers the `external.py` dispatcher. This dispatcher spawns the simulation processes using the NetQASM SDK, passing the specific rank to each instance. For example, executing the code with 3 nodes results in:

```
$ netqmpi -n 3 script.py
Hello, rank=0 of 3 processes
Hello, rank=2 of 3 processes
Hello, rank=1 of 3 processes
```

As illustrated in Figure 6, the execution flow is orchestrated by `external.py`. It parses the unified script, defines the network roles based on the requested size, and injects the corresponding rank information into the entry point of the application. Finally, it hands over the configuration to the underlying NetQASM SDK, which manages the physical spawning of the runtime processes (e.g., Alice, Bob, Charlie).

2) Automated Network Initialization

The core abstraction that enables this connectivity is the `QMPICommunicator` class, defined in the `communicator` package.

Conceptually, a communicator acts as a virtualization layer that groups a set of logical processes connected through a logical network. This construct abstracts the underlying com-

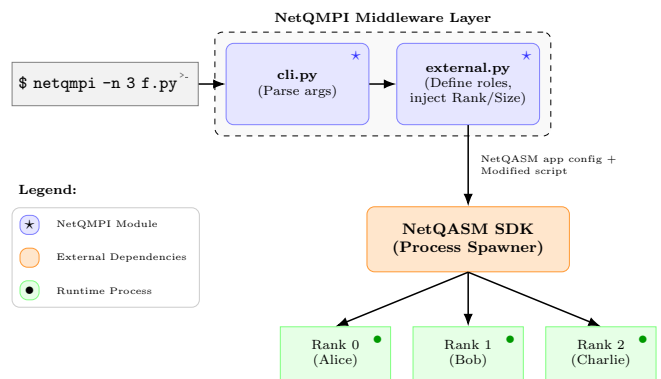


FIGURE 6. Execution flow of a NetQMPI application. The `external.py` module orchestrates the multiprocess environment, injecting rank information into the unified user script and delegating the process management to the NetQASM SDK.

plexity of the infrastructure, where logical processes (identified by ranks) are mapped to physical resources (Quantum Processing Units) via a physical interconnection network. This relationship is visually described in Figure 7. Figure 7(a) depicts the physical reality where QPUs are connected via a Quantum Router, while Figure 7(b) illustrates how the software stack (OS/Middleware) maps the high-level Logical Ranks ($1 \dots n$) to these physical resources transparently.

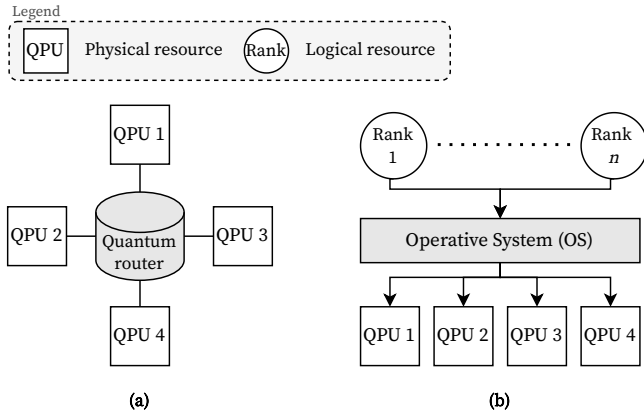


FIGURE 7. Logical abstraction for scheduling physical resources. (a) Example of physical resources interconnected between a physical quantum network. (b) The Communicator provides a logical layer where abstract Ranks are assigned to processes, and the operative system automatically handles the mapping to the available physical QPUs.

This approach mirrors the architecture of classical distributed computing, allowing developers to focus on the algorithmic logic rather than the physical topology. For a comprehensive definition of this layered abstraction model in distributed quantum computing tools, we refer the reader to the architectural framework and specifically Figure 6 presented in [12].

From an implementation perspective, the communicator automates the resource management that is manually handled in the raw SDK. In standard NetQASM, establishing a network requires the user to instantiate a `NetQASMConnection` and define specific `EPRSocket` objects for every pair of communicating nodes, leading to an $O(N^2)$ initialization complexity in fully connected topologies. NetQMPI resolves this by performing the following automated steps during initialization:

- 1) It assigns a unique rank to each logical process.
- 2) It iterates through the network configuration and automatically instantiates the necessary mesh of `EPRSocket` between all participating ranks.
- 3) It stores these sockets in an internal routing table within the communicator instance, making the management of entanglement resources transparent to the user.

Furthermore, NetQMPI handles the synchronization of the quantum control flow. While the raw SDK requires manual `flush()` invocations, NetQMPI primitives include implicit

flushing mechanisms or offer explicit `flush()` operations to ensure state consistency across the network.

C. POINT TO POINT FUNCTIONS

The fundamental requirement of a distributed quantum network is the ability to transfer quantum states between nodes. NetQMPI encapsulates the standard Quantum Teleportation protocol into two atomic primitives: `qsend` and `qrecv`.

Crucially, these primitives represent a shift towards **high-level semantic communication**. From the programmer's perspective, the operation is defined simply as sending a specific qubit to a destination rank. All references to low-level hardware resources—such as explicit EPR pair generation, entanglement fidelity, specific QPU identifiers, or classical control signaling—are completely abstracted away. The developer focuses solely on the logical flow of quantum information.

1) The `qsend` Primitive

The `qsend(qubit, dest_rank)` function abstracts the complex role of the sender ("Alice"). When this semantic function is invoked, the library transparently handles the following underlying operations:

- 1) Looks up the pre-established `EPRSocket` associated with the logical `dest_rank` in the routing table.
- 2) Requests the creation of an EPR pair from the network layer.
- 3) Performs the local Bell measurement (CNOT and Hadamard gates) between the user's payload qubit and the entangled qubit.
- 4) Measures the qubits and automatically transmits the classical measurement outcomes (2 bits) to the destination rank using the classical control plane.

2) The `qrecv` Primitive

The `qrecv(source_rank)` function abstracts the role of the receiver ("Bob"). It returns a handle to the received quantum state, allowing the program to continue execution immediately. Internally, the library:

- 1) Waits for the hardware confirmation of the entanglement generation.
- 2) Listens for the classical correction bits sent by `source_rank`.
- 3) Applies the necessary Pauli-X and Pauli-Z corrections to recover the original state fidelity.

This encapsulation drastically simplifies the development process. As shown in the example below, the complex coordination required by the raw SDK (discussed in Section II) is reduced to intuitive, single-line commands:

D. COLLECTIVE OPERATIONS

A significant advantage of the MPI paradigm is the ability to perform collective operations involving multiple nodes simultaneously. While point-to-point communication allows for specific data transfer, collective primitives abstract common data movement patterns, reducing code verbosity and

```

1 def main(app_config=None, rank, size):
2     # Initialize the communicator
3     comm = QMPIOCommunicator(rank, size, app_config)
4
5     if rank == 0:
6         # Sender Logic (Alice)
7         q = Qubit(comm.connection)
8         q.H() # Prepare state |+>
9
10        # Semantic send: "Send qubit q to Rank 1"
11        # No EPR management required
12        comm.qsend(q, 1)
13        print("Rank 0: Qubit sent")
14
15    elif rank == 1:
16        # Receiver Logic (Bob)
17        # Semantic receive: "Get qubit from Rank 0"
18        q_received = comm.qrecv(0)
19
20        # The qubit is ready for use
21        m = q_received.measure()
22        print(f"Rank 1: Qubit received, measure: {m}")

```

FIGURE 8. Example of point to point code using the `qsend` and `qrecv` primitives of NetQMPI.

potential errors. NetQMPI implements these operations by leveraging the underlying point-to-point primitives.

1) Scatter and Gather

The `qscatter` and `qgather` primitives allow for the distribution and collection of quantum states among a group of processes.

The `qscatter` operation takes a list of qubits residing in a source process (root) and distributes them sequentially to all other processes in the communicator. Conversely, `qgather` collects qubits from all processes and moves them to a destination register in the root process. From an implementation standpoint, these operations are abstractions of a loop of point-to-point transfers. For instance, a `qscatter` is functionally equivalent to the root node executing a `qsend` inside a `for` loop targeting each rank in the communicator.

Figure 9 illustrates the movement of qubits during these operations. In Figure 9(a), the initial state is shown where the root holds a register of qubits. Figure 9(b) shows the result of a scatter operation (or the start of a gather), where each qubit has been teleported to its corresponding rank.

A practical implementation is shown in Figure 10. In this example, the root process initializes a list of qubits in the superposition state $|+\rangle$ (using Hadamard gates) and distributes one qubit to each node in the network using `qscatter`. Finally, each node measures its local qubit independently.

2) Expose and Unexpose

In classical MPI, the `MPI_Bcast` operation copies data from one process to all others. However, in quantum computing, the *No-Cloning Theorem* prohibits creating independent copies of an arbitrary unknown quantum state. Therefore, a direct quantum broadcast is physically impossible.

To address this, NetQMPI introduces the `expose` and `unexpose` primitives, as proposed in the architectural framework cited [12]. Instead of copying the state, the

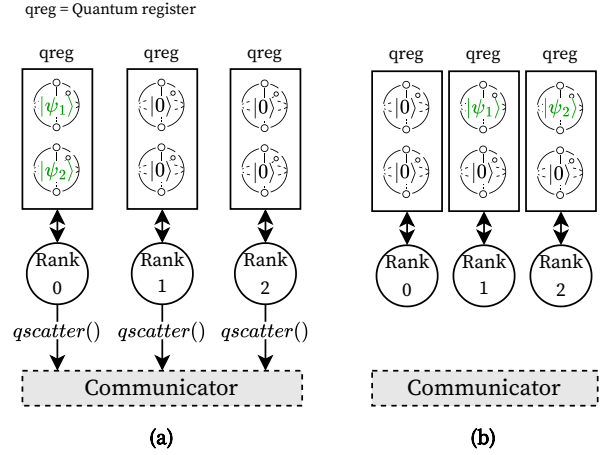


FIGURE 9. Visual representation of the `qscatter` operation. (a) Initial state where all qubits reside in the root process. (b) Distributed state where qubits have been moved to their respective ranks. The operations rely on iterative point-to-point teleportations.

```

1 def main(app_config=None, rank, size):
2     comm = QMPIOCommunicator(rank, size, app_config)
3     conn = comm.connection
4     ROOT_RANK = 0
5     qubits = []
6
7     # 1. Root initializes the qubits
8     if rank == ROOT_RANK:
9         qubits = [Qubit(conn) for _ in range(size)]
10        for q in qubits:
11            q.H() # Apply Hadamard
12
13    conn.flush()
14
15    # 2. Scatter: Distribute from Root to all
16    # Returns the specific qubit assigned to this rank
17    local_qs = comm.qscatter(qubits, ROOT_RANK)
18
19    m = local_qs[0].measure()
20    conn.flush()
21
22    print(f"Rank {rank}: measured {m}")

```

FIGURE 10. Example of the `qscatter` collective primitive in NetQMPI. The root process initializes a list of qubits in the superposition state $|+\rangle^{\otimes n}$, scatters them to all ranks, and each rank measures its local qubit.

`expose` operation creates a shared context where a specific qubit (or set of qubits) becomes conceptually accessible to other processes for collective operations, without violating the no-cloning principle.

Mathematically, the objective of `expose` is to transition the system from an initial state where the root process R_0 holds an arbitrary superposition

$$|\psi\rangle_{\text{initial}} = \alpha|0\rangle_{R_0} + \beta|1\rangle_{R_0}, \quad (1)$$

to a global state where this quantum information is distributed across all n participating ranks ($R_0, \dots, R_n$).

$$|\psi\rangle_{\text{exposed}} = \alpha(|0\rangle_{R_0} \otimes \dots \otimes |0\rangle_{R_n}) + \beta(|1\rangle_{R_0} \otimes \dots \otimes |1\rangle_{R_n}) \quad (2)$$

This shared state enables collective controlled operations on the quantum information distributed across all nodes. The

implementation relies on entangling the root qubit with a pre-shared GHZ state across all nodes, followed by local measurements and classical corrections to distribute the quantum information, as illustrated in Figure 11.

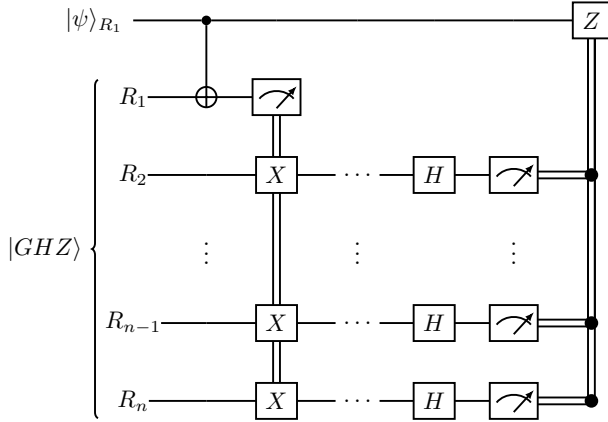


FIGURE 11. Quantum circuit representation of the `expose` and `unexpose` operation. The root qubit is entangled with a pre-shared GHZ state across all nodes, followed by local measurements and classical corrections to distribute the quantum information.

Crucially, because this operation alters the accessibility scope of the qubits, it is mandatory to release the resource once the collective computation is finished. The `unexpose` primitive serves this purpose, returning the system to a consistent state where the qubits are once again private to their owner or properly measured.

IV. COMPARATIVE AND USE CASES

To evaluate the effectiveness of the proposed abstractions, we perform a comparative analysis against the current state-of-the-art tools for distributed quantum programming. We selected the generation of a distributed Greenberger–Horne–Zeiling (GHZ) state across N nodes as the benchmark scenario. This task is chosen because it requires multi-partite entanglement and coordinated control flow, serving as a standard "Hello World" for the Quantum Internet.

A. BENCHMARK DEFINITION

The goal is to generate the maximally entangled state $|GHZ\rangle = \frac{1}{\sqrt{2}}(|00\dots 0\rangle + |11\dots 1\rangle)$ shared among N distinct network nodes.

As shown in Figure 12, the protocol involves the following steps:

- 1) One root node (e.g., Rank 0) prepares a qubit in superposition ($|+\rangle$).
- 2) The root node performs a sequence of distributed CNOT operations targeting the qubits of the remaining $N - 1$ nodes.
- 3) In a distributed setting without shared memory, every "Remote CNOT" implies a complex teleportation protocol or entanglement swapping routine involving classical communication and corrections.

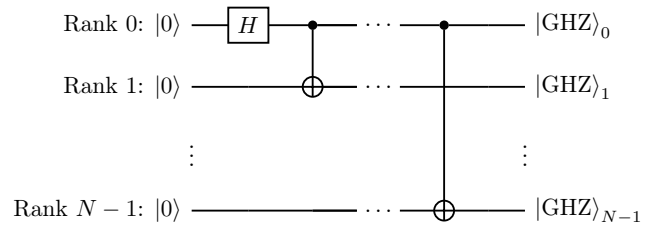


FIGURE 12. Distributed GHZ State Generation across N nodes. The root node (Rank 0) prepares a qubit in superposition and performs distributed CNOTs to entangle the qubits at the target nodes (Ranks 1 $\dots$ $N - 1$). Each Remote CNOT involves entanglement distribution and classical communication.

B. QUALITATIVE ANALYSIS: PROGRAMMING COMPLEXITY

We compare the implementation workflow in NetQMPI against three categories of tools: High-level Simulators (QuNetSim), Raw SDKs (NetQASM/SquidASM), and Discrete-Event Engines (NetSquid/SeQuENCe).

1. NetQMPI (This Work): Following the MPI standard, the solution is written in a *single source file*. The logic scales automatically with the number of processes. The developer uses the `rank` variable to distinguish between the root (Control) and the others (Targets). The distributed CNOT is abstracted, or the `expose` primitive is used to create the shared state in a single call. *Complexity:* $O(1)$ in terms of file management. The code remains constant regardless of N .

2. NetQASM SDK (SquidASM / SimulaQron): As these tools operate on a role-based paradigm, the developer must explicitly define the behavior for each node. For a 3-node GHZ state, this requires writing three separate application flows (e.g., `app_alice.py`, `app_bob.py`, `app_charlie.py`). Furthermore, a Remote CNOT is not a native primitive. The developer must manually implement the "telegate" protocol: generating EPR pairs, performing local measurements, sending classical bits via a separate socket, and applying corrections. *Complexity:* $O(N)$ in terms of source files and code maintenance. Extending the system from 3 to 4 nodes requires writing a completely new script for the new node and updating the root script to handle the new connection.

3. NetSquid / SeQuENCe: These are discrete-event simulators, not programming frameworks. Before implementing the GHZ logic, the user must manually construct the network topology in Python: defining quantum memories, fiber lengths, photon sources, and noise models. The algorithm itself is then defined as a protocol class attached to these physical entities. *Complexity:* Extremely High. The code is dominated by hardware definitions rather than algorithmic logic.

C. CODE IMPLEMENTATION EXAMPLES

To substantiate the complexity analysis, we present code snippets required to implement the GHZ state generation across the evaluated platforms.

1) NetQMPI (SPMD Paradigm)

Listing 1 shows the full implementation in NetQMPI. The logic is defined in a single block. The `expose` primitive automatically handles the underlying distributed CNOTs and entanglement distribution to create the shared state defined in Eq. 2. Changing the size from 3 to 50 nodes only requires changing the command line argument `-n 50`.

```
1 def main(app_config=None, rank=0, size=3):
2     comm = QMPIOCommunicator(rank, size, app_config)
3
4     # 1. Root creates the base qubit
5     qubits = []
6     q = Qubit(comm.connection)
7     if rank == 0:
8         q.H() # Superposition
9         qubits.append(q)
10
11    # 2. Collective Operation
12    # The root exposes the qubit to all ranks (control of
13    # remote CNOT)
14    comm.expose(qubits, root_rank=0)
15
16    # 3. Perform local operations (if my rank is not root
17    # )
18    if rank != 0:
19        qubits[0].cnot(my_qubit)
20
21    comm.unexpose(root_rank=0)
```

Listing 1. NetQMPI: GHZ Generation (Single File)

2) NetQASM SDK (Role-Based)

In the raw SDK, the logic is strictly separated by roles. Listing 2 demonstrates the code required to generate a GHZ state among three nodes (Alice, Bob, and Charlie).

Crucially, note that the developer must explicitly instantiate the connection objects (EPRSocket for quantum links and Socket for classical communication) *before* the main logic. This reveals the scalability bottleneck: adding a third node (Charlie) forced us to:

- 1) Create a completely new file `app_charlie.py`.
- 2) Modify `app_alice.py` to manually instantiate the new sockets for Charlie (`sock_charlie`, `csock_charlie`) and duplicate the teleportation logic.

```
1 # --- app_alice.py (Root) ---
2 # 1. Explicit Socket Creation
3 # Alice needs separate sockets for each target
4 sock_bob = EPRSocket("Bob")
5 sock_charlie = EPRSocket("Charlie")
6 csock_bob = Socket("Alice", "Bob") # Classical
7 csock_charlie = Socket("Alice", "Charlie") # Classical
8
9 with NetQASMConnection("Alice", epr_sockets=[sock_bob,
10 sock_charlie]) as c:
11     q = Qubit(c)
12     q.H()
13
14    # 2. Manual Remote CNOT to Bob
15    epr_b = sock_bob.create_epr()[0]
16    q.cnot(epr_b)
17    m_b = epr_b.measure()
18    c.flush()
19    csock_bob.send(str(m_b)) # Send correction bits
20
21    # 3. Manual Remote CNOT to Charlie
22    # Logic must be duplicated for the new node
23    epr_c = sock_charlie.create_epr()[0]
24    q.cnot(epr_c)
```

```
24 m_c = epr_c.measure()
25 c.flush()
26 csock_charlie.send(str(m_c))
27
28 # --- app_bob.py (Target 1) ---
29 sock_alice = EPRSocket("Alice")
30 csock_alice = Socket("Bob", "Alice")
31
32 with NetQASMConnection("Bob", epr_sockets=[sock_alice])
33 as c:
34     q = Qubit(c)
35     epr = sock_alice.recv_epr()[0]
36     epr.cnot(q)
37     c.flush()
38
39 # --- app_charlie.py (Target 2) ---
40 # New file required for the third node
41 sock_alice = EPRSocket("Alice")
42 csock_alice = Socket("Charlie", "Alice")
43
44 with NetQASMConnection("Charlie", epr_sockets=[sock_alice
45 ]) as c:
46     q = Qubit(c)
47     epr = sock_alice.recv_epr()[0]
48     epr.cnot(q)
49     c.flush()
```

Listing 2. NetQASM SDK: GHZ Implementation (Alice, Bob, Charlie)

3) NetSquid (Discrete-Event / Hardware)

NetSquid operates at a lower abstraction level. Before implementing the GHZ protocol, the user must define the physical hardware. Listing 3 illustrates the verbosity of setting up just the nodes and channels. The actual algorithm requires defining a class inheriting from `NodeProtocol` for each entity.

```
1 # 1. Define Hardware Components
2 qproc_alice = Node("Alice", num_positions=2,
3 mem_noise_models=...)
4 qproc_bob = Node("Bob", num_positions=2, ...)
5
6 # 2. Define Physical Connections
7 channel_a_b = QuantumChannel("A->B", length=20,
8 models={"delay_model": FibreDelayModel()})
9
10 # 3. Network Configuration
11 network = Network("GHZ_Net")
12 network.add_node(qproc_alice)
13 network.add_connection(qproc_alice, qproc_bob,
14 channel_to=channel_a_b)
15
16 # 4. Protocol Implementation (Not shown: ~100 lines)
17 # Requires defining state machines for photon emission
```

Listing 3. NetSquid: Hardware Setup

4) QuNetSim (High-Level Simulator)

QuNetSim provides a centralized view similar to a network orchestrator. While the algorithmic part is concise, it requires significant configuration code to set up the simulation environment. The developer must manually instantiate hosts, configure the network topology, establish connections, and manage the start/stop lifecycle of the simulation engine (see Listing 4). Furthermore, this code runs purely as a simulation and cannot be compiled to real hardware instructions.

```
1 # 1. Manual Network Setup
2 network = Network.get_instance()
3 alice = Host('Alice')
4 bob = Host('Bob')
5 charlie = Host('Charlie')
```

```

6
7 # 2. Manual Connection Management
8 alice.add_connection('Bob')
9 alice.add_connection('Charlie')
10 bob.add_connection('Charlie') # ... and others ...
11 alice.start(); bob.start(); charlie.start()
12 network.start()
13
14 # 3. Protocol: Manual Circuit Construction
15 def ghz_protocol_alice(host):
16     # a. Create Root Qubit
17     q_root = Qubit(host)
18     q_root.H()
19
20     # b. Entangle and Send to Bob
21     q_bob = Qubit(host)
22     q_root.cnot(q_bob) # Local CNOT
23
24     host.send_qubit('Bob', q_bob)
25
26     # c. Entangle and Send to Charlie
27     q_charlie = Qubit(host)
28     q_root.cnot(q_charlie)
29     host.send_qubit('Charlie', q_charlie)
30
31     # ... Wait for acknowledgments ...
32
33 def ghz_protocol_target(host):
34     # Targets must explicitly receive the qubit
35     q = host.get_data_qubit('Alice')
36     print(f"{host.host_id} received part of GHZ")
37
38 # 4. Execution
39 t1 = alice.run_protocol(ghz_protocol_alice)
40 t2 = bob.run_protocol(ghz_protocol_target)
41 t3 = charlie.run_protocol(ghz_protocol_target)
42 t1.join(); t2.join(); t3.join()
43 network.stop(stop_hosts=True)

```

Listing 4. QuNetSim: Network Setup & Orchestration

D. QUANTITATIVE ANALYSIS: LINES OF CODE (LOC)

To evaluate the development effort and scalability of the proposed solution, we analyzed the implementation of the GHZ generation protocol across the selected platforms. We utilize lines of code (LOC) as a metric for software complexity, counting only logical lines (excluding comments and imports).

Table 1 summarizes the asymptotic complexity required for both setting up the network topology and implementing the computational logic.

Framework	Files	LOC (Network)	LOC (Comp.)	Paradigm
NetQMPI	1	$\mathcal{O}(1)$	$\mathcal{O}(1)$	SPMD (Abstract)
QuNetSim	1	$\mathcal{O}(N^2)$	$\mathcal{O}(1)$	Centralized Sim
NetQASM SDK	N	$\mathcal{O}(N^2)$	$\mathcal{O}(N)$	Role-based
NetSquid	N	$\mathcal{O}(N^2)$	$\mathcal{O}(N)$	Hardware Descr.

TABLE 1. Comparison of Lines of Code (LOC) complexities required for setting up the network topology and implementing the GHZ generation as the number of nodes N increases.

Trend Analysis and Scalability: The projection of code growth as the network size N increases is illustrated in Figure 13. The analysis reveals three distinct trends:

- **Linear/Quadratic Growth (State of the Art):** In frameworks like the NetQASM SDK or NetSquid, the

complexity grows significantly with N . Each new node requires the creation of separate source files, manual instantiation of connection sockets ($\mathcal{O}(N^2)$ for fully connected topologies), and duplication of control logic. This verbose approach forces the developer to write boilerplate code that is essentially identical but structurally necessary, dramatically increasing the probability of introducing copy-paste errors or synchronization bugs.

- **Constant Trend (NetQMPI):** In contrast, NetQMPI maintains a flat complexity curve ($\mathcal{O}(1)$). Thanks to the SPMD paradigm and collective primitives, the code written for $N = 3$ is identical to the code for $N = 100$. The logic is encapsulated in loops (e.g., `expose`) that iterate over the communicator size dynamically.

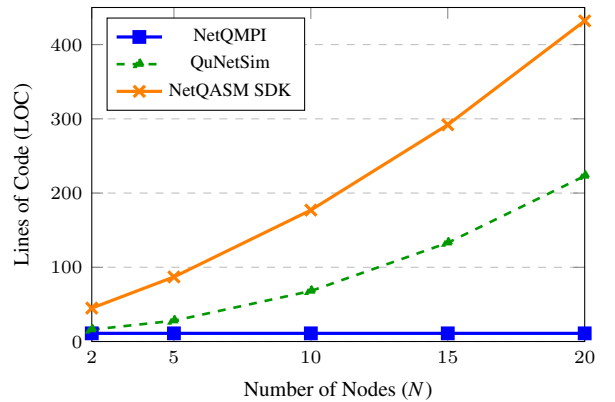


FIGURE 13. Comparison of Lines of Code (LOC) required to implement the GHZ state generation protocol as the network size (N) increases. NetQMPI maintains constant complexity due to its SPMD collective operations.

Implications for Quantum Software Engineering: This reduction in code complexity is critical for the development of distributed quantum applications. By decoupling the program logic from the network size, NetQMPI allows for rapid prototyping and ensures a bug-free implementation of complex protocols. In a domain where debugging distributed quantum states is notoriously difficult, minimizing the codebase surface reduces the potential for implementation errors, allowing researchers to focus on algorithmic correctness rather than infrastructure management.

V. CONCLUSIONS

This work has introduced NetQMPI, a high-level software framework designed to bridge the gap between abstract quantum algorithm design and the low-level management of distributed quantum network resources. By abstracting the verbose primitives of the NetQASM SDK, NetQMPI successfully reduces the complexity of programming distributed quantum applications, offering a scalable and robust development environment.

The primary conclusions extracted from this work are summarized as follows:

- **Paradigm SPMD:** NetQMPI facilitates a transition from the labor-intensive, role-based programming

model of existing SDKs to a Single Program Multiple Data (SPMD) paradigm. By unifying the control logic into a single source file where roles are defined by the process rank, we have demonstrated that the complexity of setting up a quantum network drops from $\mathcal{O}(N^2)$ to $\mathcal{O}(1)$ in LOC number. This decoupling of program logic from network size is a fundamental requirement for scalable quantum software engineering.

- **Abstraction of the Network Layer:** The library introduces the `QMPICommunicator` and semantic point-to-point primitives (`qsend`, `qrecv`) that completely hide the physical layer's intricacies. The developer is no longer required to manually manage `EPRSocket` life-cycles, entanglement fidelity, or the synchronization of classical control signals, allowing them to focus entirely on the algorithmic flow of quantum information.
- **Implementation of Collective Operations:** We have provided practical implementations for collective communication in a quantum context. Notably, the introduction of the `expose` and `unexpose` primitives addresses the physical constraints imposed by the No-Cloning Theorem. These operations allow for the creation of shared multipartite entangled contexts (such as GHZ states) to simulate broadcasting behavior without violating quantum mechanical principles.
- **Integration with the quantum ecosystem:** A critical objective of NetQMPI is its seamless integration with the existing quantum stack. By compiling down to the standardized NetQASM ISA, NetQMPI acts as a backend-agnostic middleware. This allows researchers to execute high-level code on state-of-the-art simulators like **SquidASM** and **NetSquid**, leveraging their rigorous physical noise models without facing their steep learning curves. Furthermore, this architecture ensures that applications written today in NetQMPI will be executable on future physical quantum hardware that supports the NetQASM standard.

In summary, NetQMPI represents a significant step forward in quantum software engineering. It allows developers to write readable, maintainable, and bug-free distributed quantum applications and paving the way for the implementation of complex distributed algorithms.

REFERENCES

- [1] H. Buhrman, R. Cleve, and A. Wigderson, "Quantum vs. Classical Communication and Computation," *Conference Proceedings of the Annual ACM Symposium on Theory of Computing*, pp. 63–68, 2 1998. [Online]. Available: <https://arxiv.org/pdf/quant-ph/9802040>
- [2] I. L. Markov, "Limits on fundamental limits to computation," *Nature* 2014 512:7513, vol. 512, no. 7513, pp. 147–154, 8 2014. [Online]. Available: <https://www.nature.com/articles/nature13570>
- [3] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Review*, vol. 41, no. 2, pp. 303–332, 1999. [Online]. Available: <https://epubs.siam.org/doi/10.1137/S0097539795293172>
- [4] L. K. Grover, "A fast quantum mechanical algorithm for database search," *Proceedings of the Annual ACM Symposium on Theory of Computing*, vol. Part F129452, pp. 212–219, 7 1996. [Online]. Available: <https://dl.acm.org/doi/pdf/10.1145/237814.237866>
- [5] J. Preskill, "Quantum Computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 8 2018. [Online]. Available: <https://quantum-journal.org/papers/q-2018-08-06-79/>
- [6] D. Barral, F. J. Cardama, G. Díaz-Camacho, D. Faílde, I. F. Llovo, M. Mussa-Juane, J. Vázquez-Pérez, J. Villasuso, C. Piñeiro, N. Costas, J. C. Pichel, T. F. Pena, and A. Gómez, "Review of Distributed Quantum Computing: From single QPU to High Performance Quantum Computing," *Computer Science Review*, vol. 57, p. 100747, 8 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1574013725000231>
- [7] M. Caleffi, M. Amoretti, D. Ferrari, J. Illiano, A. Manzalini, and A. S. Cacciapuoti, "Distributed quantum computing: A survey," *Computer Networks*, vol. 254, p. 110672, 12 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1389128624005048?via%3Dihub>
- [8] R. Van Meter, W. J. Munro, and K. Nemoto, "Architecture of a Quantum Multicomputer Implementing Shor's Algorithm," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 5106 LNCS, pp. 105–114, 2008. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-540-89304-2_10
- [9] A. Ovide, S. Rodrigo, M. Bandic, H. Van Someren, S. Feld, S. Abadal, E. Alarcon, and C. G. Almudever, "Mapping quantum algorithms to multi-core quantum computing architectures," *Proceedings - IEEE International Symposium on Circuits and Systems*, vol. 2023-May, 2023. [Online]. Available: <https://ieeexplore.ieee.org/document/10181589>
- [10] S. Wehner, D. Elkouss, and R. Hanson, "Quantum internet: A vision for the road ahead," *Science*, vol. 362, no. 6412, 10 2018. [Online]. Available: <https://www.science.org/doi/10.1126/science.aam9288>
- [11] S. W. Loke, "From Distributed Quantum Computing to Quantum Internet Computing: an Overview," *arXiv*, 8 2022. [Online]. Available: <https://arxiv.org/pdf/2208.10127>
- [12] F. J. Cardama, J. Vázquez-Pérez, C. Piñeiro, T. F. Pena, J. C. Pichel, and A. Gómez, "NetQIR: An extension of QIR for distributed quantum computing," *Future Generation Computer Systems*, vol. 174, p. 107989, 1 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X25002845>
- [13] G. Díaz-Camacho, I. F. Llovo, F. J. Cardama, I. Bautista, D. Faílde, M. M. Juane, J. Vázquez-Pérez, N. Costas, T. F. Pena, and A. Gómez, "Benchmarking Distributed Quantum Computing Emulators," *arXiv*, 12 2025. [Online]. Available: <https://arxiv.org/pdf/2512.01807>
- [14] T. Haner, D. S. Steiger, T. Hoefler, and M. Troyer, "Distributed quantum computing with QMPI," *International Conference for High Performance Computing, Networking, Storage and Analysis, SC*, 11 2021. [Online]. Available: <https://dl.acm.org/doi/pdf/10.1145/3458817.3476172>
- [15] R. Parekh, A. Ricciardi, A. Darwish, and S. Diadamo, "Quantum Algorithms and Simulation for Parallel and Distributed Quantum Computing," *Proceedings of QCS 2021: 2nd International Workshop on Quantum Computing Software, Held in conjunction with SC 2021: The International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 9–19, 2021. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9651415>
- [16] S. Diadamo, J. Nötzel, B. Zanger, and M. M. Beşe, "QuNetSim: A Software Framework for Quantum Networks," *IEEE Transactions on Quantum Engineering*, vol. 2, 2021. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9465750>
- [17] A. Dahlberg and S. Wehner, "SimulaQron—a simulator for developing quantum internet software," *Quantum Science and Technology*, vol. 4, no. 1, p. 015001, 9 2018. [Online]. Available: <https://iopscience.iop.org/article/10.1088/2058-9565/aad56e>
- [18] A. Dahlberg, B. Van Der Vecht, C. Delle Donne, M. Skrzypczyk, I. Te Raa, W. Kozłowski, and S. Wehner, "NetQASM—a low-level instruction set architecture for hybrid quantum-classical programs in a quantum internet," *Quantum Science and Technology*, vol. 7, no. 3, p. 035023, 6 2025. [Online]. Available: <https://iopscience.iop.org/article/10.1088/2058-9565/ac753f>
- [19] T. Coopmans, R. Knegjens, A. Dahlberg, D. Maier, L. Nijsten, J. de Oliveira Filho, M. Papendrecht, J. Rabbie, F. Rozpędek, M. Skrzypczyk, L. Wubben, W. de Jong, D. Podareanu, A. Torres-Knoop, D. Elkouss, and S. Wehner, "NetSquid, a NETwork Simulator for QUantum Information using Discrete events," *Communications Physics* 2021 4:1, vol. 4, no. 1, pp. 164–, 7 2021. [Online]. Available: <https://www.nature.com/articles/s42005-021-00647-8>

- [20] X. Wu, A. Kolar, J. Chung, D. Jin, T. Zhong, R. Kettimuthu, and M. Suchara, "SeQUeNCe: a customizable discrete-event simulator of quantum networks," *Quantum Science and Technology*, vol. 6, no. 4, p. 045027, 9 2021. [Online]. Available: <https://iopscience.iop.org/article/10.1088/2058-9565/ac22f6>



F. JAVIER CARDAMA (Student Member) received the B.S. degree in Computer Engineering from the University of Santiago de Compostela (USC), Spain, in 2021, and the M.S. degree in High-Performance Computing from the same university in 2022. He is currently pursuing the Ph.D. degree in Information Technology Research at the Centro Singular de Investigación en Tecnoloxías Intelixentes (CiTIUS), USC, holding a Xunta de Galicia fellowship. His research interests focus on

distributed quantum computing and high-performance computing, specifically on the development of software frameworks and architectures for the integration of quantum technologies in HPC centers.



TOMÁS F. PENA (Senior Member) received the Ph.D. degree in physics from the University of Santiago de Compostela (USC), Santiago, Spain, in 1994. He is currently an Full Professor with the Department of Electronics and Computer Science, USC, and a Member of the Research Center in Intelligent Technologies (CiTIUS), Santiago de Compostela, Spain. His research interests include distributed quantum computing, high-performance computing, parallel systems architec-

ture, optimization of performance in irregular codes and with sparse matrices, and Big Data technologies.

...